

Code Generation (ER → SQL)

Blaž Bone · Tilen Ožbot

June 2026

Supported ER Concepts — Entities & Relationships

Concept	T	G	Gen
Strong entity	✓	✓	✓
Weak entity	✓	✓	✓
Binary rel.	✓	✓	✓
Unary rel.	✓	✓	⬢
Ternary rel.	✓	✓	✓
N-ary (≥ 4)	✗	✗	⬢
Cardinality	✓	✓	⬢
Participation	✓	✓	✗
Roles	✓	✓	⬢

Supported ER Concepts — Attributes & EER

Concept	T	G	Gen
Entity attr.	✓	✓	✓
Rel. attr.	✓	◐	✓
Keys	✓	✓	✓
Datatypes	✓	✓	✓
Composite	✗	✗	✗
Multivalued	✓	✓	✓
Derived	✓	✓	✓
Generalization	✓	✓	✓

T = textual · G = graphical · Gen = generator · ✓ supported · ◐ partial · ✗ unsupported

Assumptions — Types and Constraints

- Datatype mapping
- Nullability — NOT NULL by default
- Cardinality — UNIQUE from look-across constraints
- Aggregation and composition

Assumptions — Relational Mapping

- Weak entity → folded into one table with owner FK
- Generalization — class-table inheritance
- Multi-valued attribute → side table
- Roles → FK column-name prefix for self-references

Assumptions — Export Behavior

- `generateDrop` flag → reverse-emission `DROP` order
- `DROP TABLE IF EXISTS` is dialect-agnostic
- Parent entities are declared before subclasses

Testing — What We Test

.er file (input) → Generator (our code) → SQL output (Postgres / MySQL)

Stage	Question it answers
0. Coverage	Does every fixture have both dialect goldens?
1. Grammar	Does the golden SQL parse as valid syntax?
2. Golden match	Does the generator produce exactly the SQL we wrote?
3. Engine	Does the SQL run on a real database?
4. Cross-dialect	Do Postgres and MySQL schemas agree?
Behavioural	Do constraints actually reject bad inserts?
Property	Is the generator deterministic on random inputs?

Testing — How It Works

15 fixtures × 2 dialects = 30 test scenarios

Each fixture:	foo.er	← ER model
	foo.postgres.sql	← hand-written expected SQL
	foo.mysql.sql	← hand-written expected SQL

Stage 1: Parse .sql with real parsers (libpg-query / dt-sql-parser)



Stage 2: Run generator on .er → byte-for-byte match vs golden



Stage 3: Execute SQL on real Postgres (PGLite) + MySQL (Docker)



Stage 4: Snapshot both DB schemas → normalize types → compare



Behavioural: INSERT bad data → assert constraints reject it

Adding a test: drop 3 files → `yarn test` → done. Zero code changes.

Testing — The Numbers

Layer	Tests	Powered by
Coverage	1	File scanning
Grammar	30	libpg-query (WASM), dt-sql-parser
Golden match	30	Vitest <code>toBe</code> (exact string)
Engine	30	PGlite (no Docker), Testcontainers
Cross-dialect	15	information_schema + type normalization
Behavioural	10	Live INSERT / REJECT on both DBs
Property	4	fast-check (50 random models)
Unit	22	Validator checks + generateDrop option
Total	142	